



When Open Source Meets Network Control Planes

Christian Esteve Rothenberg, *University of Campinas*

Roy Chua, *SDN Central and Wiretap Ventures*

Josh Bailey, *Google*

Martin Winter, *Network Device Education Foundation*

Carlos N.A. Corrêa, *Federal Fluminense University and Unimed*

Sidney C. de Lucena, *Federal University of the State of Rio de Janeiro*

Marcos Rogério Salvador, *Lenovo Innovation Center*

Thomas D. Nadeau, *Brocade*

Software-defined networking opens up new possibilities for architectures based on open source components, promising improved orchestration and agility, lower operational costs, and—most important—a wave of innovation.

A primary goal for software-defined networking (SDN) is to introduce new abstractions that help network operators rethink and separate the distribution of control plane and data forwarding functions, which have traditionally been placed in close proximity on the same device. Successful production and experimental implementations use protocols and APIs that are swiftly becoming standards, if they are not already.^{1,2} Indeed, SDN's scope has progressively broadened since OpenFlow triggered its inception in 2009.³ Today, SDN encompasses multiple networking proposals, from OpenFlow's original control plane and data plane split² to software-based edge tunnels that define virtual network overlays to hybrids that augment and expose existing distributed protocols through programmable interfaces, such as the Internet Engineering Task Force's I2RS.⁴

Whatever its manifestation, SDN gives network architects new flexibility for using open source components. The advent of the Open Daylight Project (ODP), along with related large-scale, open source infrastructure projects such as OpenStack and OpenCompute, puts SDN in a position to replicate the success of the archetypal Web services

solution stack LAMP (Linux, Apache, MySQL, PHP). With LAMP, IT architects can build scalable server farms from low-cost commodity gear with minimal licensing costs. SDN's promise is similar: assemble a set of open source software components to define the network control logic and use open APIs to interact with multivendor networking hardware. Adaptability will be higher and ownership cost will be lower because solutions are built from high-performance commercial networking hardware and open source software running in commodity server technology.

A rich ecosystem is developing around SDN, and API availability and maturity will play a critical role in defining the networking landscape. Our experience creating and deploying production SDN architectures and operations, specifically RouteFlow⁵ and Cardigan,⁶ has given us a basis for exploring the new wave of SDN-based solutions and understanding how open source networking is evolving to unlock opportunities once available only through vendor-proprietary solutions.

Despite those who argue that SDN has seen limited deployment, projects such as Google's B4, described in the sidebar "The Case for SDN," lead us to believe that network architects will soon be using fully open source-enabled SDN stacks.

OVERVIEW OF OPEN SOURCE SDN PROJECTS

Free/libre/open source software (FLOSS) provides the networking industry with low-cost and customizable network management and configuration tools, security suites, protocol testing and interoperability packages, and software appliances. As with any software-centric

industry, the reasons behind open source's success are the customization opportunities and development and commercialization economies it affords.⁷

Until recently, open source's success in networking has been limited to the management plane and to software appliances that run in commodity x86 servers. In the latter category are software-based IP routers such as Quagga and the Bird Internet routing daemon (BIRD), and security solutions such as Bro and Snort—all of which are free, open source code. Also available are commercialized counterpart versions enhanced with value-added services.

Now, however, open source in SDN is expanding implementation possibilities.

Evolution to the open source SDN stack

Figure 1 shows the evolution from traditional networking to the open source SDN stack. The traditional networking scenario in Figure 1a is augmented in Figure 1b by SDN (OpenFlow in particular), which allows the network's brain—the control plane and related applications—to be implemented in the high-level programming languages on commodity server technology untangled from the physical boxes that implement forwarding plane functions. Theoretical SDN abstractions for forwarding and control planes have led to software-based embodiments that are contributing to the open source SDN stack in Figure 1c.

Figures 1b and 1c portray two API types, southbound and northbound. Southbound APIs facilitate communication between the SDN controller and the network devices such as switches and routers (virtualized or physical), aiming to meet real-time traffic demands. Northbound APIs, on the other hand, facilitate communication between the SDN controller and network applications that may in turn provide network services and functions to users or other applications. This affords designers of these applications the flexibility to easily and rapidly customize and innovate.

Characteristics and trends

Table 1 provides an overview of open source projects in parts of the ecosystem around the SDN stack. These projects, taken from SDNcentral.com (www.sdncentral.com/comprehensive-list-of-open-source-sdn-projects) vary in code quality and the extent of active development and community support. Many are primarily academic exercises, but a few—Open Daylight, OpenContrail, and ONOS, for example—benefit from ample financial and professional development support.

Aside from offering libraries to interact with networking devices, open source projects are actively competing to develop the SDN controller and northbound APIs—a critical step to maturing the SDN ecosystem. Control and management applications are already addressing objectives such as topology discovery and routing, but, like support tools, these existing applications are controller-specific and lack a common API to enable portability.

The Case for SDN

If SDN is such a useful advance, why is its deployment in production networks so limited? There are several reasons, including the need to directly address operational comfort with SDN, provide concrete benefits, and demonstrate a practical migration path, starting with the ability to interoperate with non-SDNs.

Skeptics may argue that notions such as redistributing control plane functions or computing offline routes already exist and just need further exploitation, so why move to SDN? However, the maturity of projects such as OpenDaylight and OpenStack and the introduction of commercial products based on those projects are proof that these new approaches are succeeding.

Recent production projects demonstrate that SDN allows simpler, more reliable, and easier network operation. Proof of that is in projects such as Google's B4 inter-datacenter WAN¹ and Microsoft's Swan.² Efforts within RouteFlow and Cardigan also show the concrete benefits of SDN while interworking with legacy routers.

B4, operational since 2012, is based on SDN principles, OpenFlow technology, and open source components, such as the Quagga routing stack. Relative to traditional distributed control planes and traffic engineering tools based on multiprotocol label switching, network operation with B4 is simpler, easier, and more reliable. Further, by augmenting the SDN routing system with custom traffic engineering algorithms, B4 attains link utilization levels of close to 100 percent, which is unprecedented. In summary, B4 is an appealing, low-cost proposition that traditional networking approaches could simply not have achieved until the introduction of these new SDN concepts.

While some might argue that B4 is an exceptional SDN deployment, coming from a major industry player with the resources and talent to release a cutting-edge internal networking project, perhaps it is less an exception than the result of simple timing: technology being adopted within a particular ecosystem maturity window.

Unlike many other open source ecosystems, SDN grew quickly in academic environments, and only recently have users had a chance to test SDN stacks in production environments. RouteFlow and Cardigan are examples of many open source projects that will start appearing in production now that SDN technologies are maturing.

References

1. S. Jain et al., "B4: Experience with a Globally-Deployed Software Defined WAN," *Proc. 2013 Ann. Conf. ACM Special Interest Group Data Comm. (SIGCOMM 13)*, 2013, pp. 3–14.
2. C. Hong et al., "Achieving High Utilization with Software-driven WAN," *Proc. 2013 Ann. Conf. ACM Special Interest Group Data Comm. (SIGCOMM 13)*, 2013, pp. 15–26.

Similar to the early days of web servers or Java application servers, competition is fostering ideas and driving innovation. In the next few years, we expect to see an increase in SDN projects (controllers, applications, testing, and integrated development environments), which is a prerequisite to arriving at a platform consensus.

OpenDaylight Project. At the controller level, noteworthy activities are underway within the ODP, a vendor-led initiative of the Linux Foundation to create an open source SDN controller and application ecosystem to counterbalance

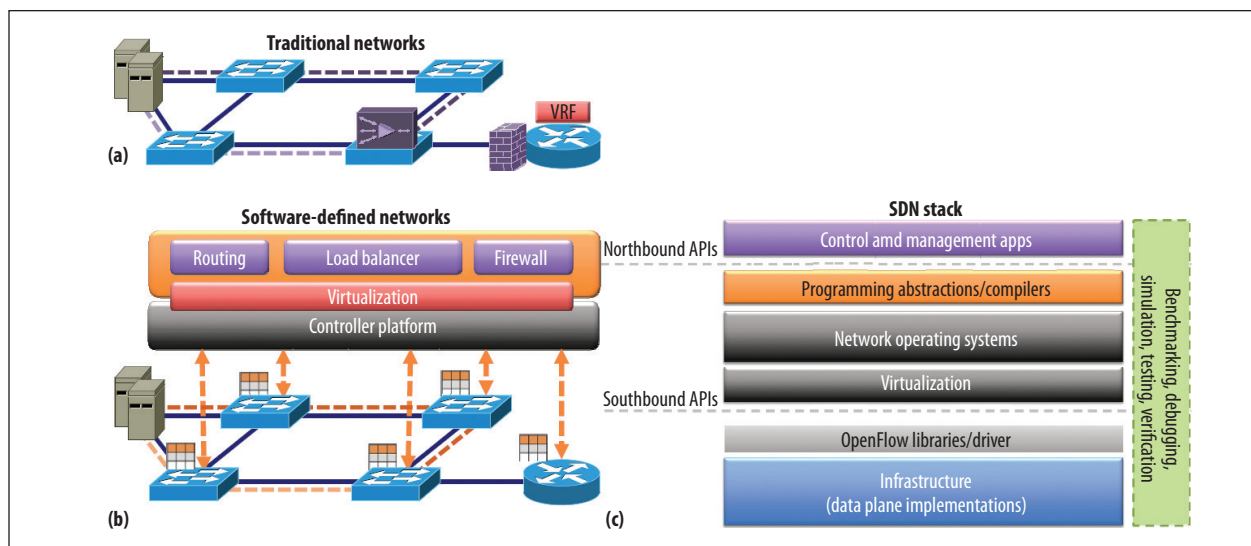


Figure 1. Path from traditional networks to an open source software-defined networking (SDN) stack. (a) Traditional networks with distributed control and device-specific management contrast with (b) SDN's flexible and logically centralized control and management, in which southbound protocols interact with data plane devices. (c) The emerging open source SDN stack enables this interaction, relying on open APIs between functional layers and on open source supporting tools, which are often highly customizable. VRF: virtual routing and forwarding.

the many vendor-proprietary or partially open source SDN controllers. One of the project's goals is to produce a commercially viable, open source-based SDN controller. The ODP incorporates other open source projects into its code base and integrates with other southbound drivers or northbound interfaces, such as OpenStack Neutron, creating further momentum toward an open source SDN controller.

Many successful open source projects start with vendor- and community-driven project management. As the projects become larger and more complex, a foundation forms that provides more structured evolution. Apache and Eclipse are examples. Historically, most successful open source projects have been community-driven, but the occasional vendor-driven project could mature if it is able to attract involvement from other communities or create a community of its own.

Community involvement generally increases the diversity of code committers and base code, both of which are integral to successful open source projects. The ODP, for example, includes coders with more than a hundred affiliations.

Although the ODP's success is still unproven, industry adoption of its controller is increasing with its second release. Many large vendors, including Brocade, Redhat, Ciena, Ericsson, and Cisco, have announced plans for commercial products based on ODP's controller, by providing their own distributions or enterprise versions with paid support plans.

Open source and standardization. As in LAMP, industry-driven open source controller projects like the ODP push functionality beyond OpenFlow, generating an ecosystem

that allows architects to snap together various components. Vendors can use a shared platform, which reduces fragmentation and user risk. In addition, vendors can benefit from one another's development efforts in areas that do not threaten strategic product differentiation.

In this context, open source becomes less a fixed solution and more a development ethos or mindset to rapidly construct and evolve tools, techniques, and running code based on community input, knowledge, and interest. Collectively, these forces will introduce a market process in which de facto standards are forged through open source platforms parallel with or complementing traditional standardization.

The characteristics in Table 2 show the contrast between traditional standardization and SDN and are evidence that SDN is accelerating standardization. Although vendors have long driven standards development, in open source SDN, users are the main force, as network operators and equipment buyers see the benefits of innovating their network infrastructure and of reducing costs and vendor lock-in.

State of APIs

APIs from open source community development should help decrease the time to market for proofs of concept and products, which in turn should shorten the path to working standards. Thus, API standardization (at least de facto standardization) and maturation are key for open source SDN. Southbound APIs are more mature overall, but their heterogeneity introduces obstacles. Northbound APIs are less mature and thus less defined, which implies possible flux

Table 1. Active open source SDN projects.

SDN realm	Project name
Benchmarking	Cbench (GPLv2, C, Perl, UNIX shell), OFLOPS (GPL, C)
Debugging, testing, and simulation	ndb, OFRewind, STS (Apache, Python), OFDissector (BSD, C), liboftrace (BSD, C), OFTest (BSD, Python), Mininet (BSD, Python), fs-sdn (GPL, Python), ns-3 (GPL, C++), TestON (GPL, Python)
Verification	Hassel (GPL, Python), NetPlumber (GPL, Python), NICE (BSD, Python), FlowChecker, OFTEN
Control and management applications	Topology discovery, (GPL, many), HostTracker (GPL, many), Plug-n-Serve (BSD, C++), Aster*x (BSD, C++), FlowScale (Apache, Java), SNAC (Python, C++), Odin (Apache, Python), PANE (BSD, Haskell), FRESCO (GPL, Python, C++), OSCARS (BSD, Java), RouteFlow (Apache, Python, C, C++, Java), Open DayLight (Eclipse, Java)
Programming abstractions, compilers, and isolation	FatTire, Flog, FML, Frenetic (GPL, OCaml), HFT, NetCore, Nettle, Procera, Pyretic (BSD, Python), Maple (Python), FlowN, LibNetVirt (GPL, C, Python), OpenStack Neutron (Apache, Python)
Controller platforms	NOX (GPL, C++), POX (GPL, Python), Maestro (LGPL, Java), Beacon (GPL, Java), Floodlight (Apache, Java), Ryu (Apache, Python), Trema (GPL, C, Ruby), Flower (MIT, Erlang), NodeFlow (GPL, javascript), Mul (GPL, C), Open Daylight (Eclipse, Java), ONOS (Java), OpenContrail (Apache, C++, Python)
Data plane virtualization	FlowVisor (BSD, Java), PortVirt (BSD, C), Expedient (Apache, Python), OpenVirteX (Apache, Java)
OpenFlow protocol libraries and southbound drivers	OFLib-Node (BSD, JavaScript), OpenFlowJ (BSD, Java), OpenFaucet (Apache, Python), Pylib-OpenFlow (BSD, Python/C++), freeflow (Apache, C, C++), xDPd, ROFL (MPL, C, C++), libfluid (Apache, C/C++)
Data plane implementations	NetFPGA (BSD, C, Verilog), Open vSwitch (Apache, C), Reference design (BSD, C), softswitch 13 (BSD, C), OpenWrt/Pantou (GPL, C), Switch Light (Eclipse, C), Indigo Virtual Switch (Eclipse, C), LINC-Switch (Apache, Erlang).

*BSD, Berkeley software distribution; FML, Flow-Based Management Language; FRESCO, Framework for Enabling Security Controls in OpenFlow (networks); GPL, GNU general public license; HFT, high-frequency trading (algorithm); LGPL, Lesser GPL; LINC, LINC Is Not Closed; MPL, Mozilla Public License; NICE, Network Interface Controller with encoder; NOX, network operating system; OF, OpenFlow; OFTEN, OpenFlow Testing Environment; ONOS, Open Network Operating System; OpenWrt, Open WRT (from Linksys WRT54G); OSCARS, OpenFlow and On-Demand Secure Circuits and Advance Reservation System; PANE, participatory networking; POX, Python network operating system; ROFL Revised OpenFlow Library set; SNAC, Simple Network Access Control; xDPd, extensible datapath daemon

Table 2. Characteristics of traditional networking versus SDN in standards development.

Characteristic	Traditional	With SDN
Drivers	Vendors	Customer
Goals	Enable multiple solutions (interoperability)	Address user or operator needs (customization)
Deliverables	Documents	Implementations and proofs of concept (PoCs)
Quantity of Standards	More	Less
Timetable	Many years	Few years
Validation	Products and deployments after release	PoCs integral to the process
Control point	Seat at standards committee table	Contribution to FLOSS codebase; ability to understand codebase
Involved parties	Vendors who can afford membership fees; highly regarded experts and academics	Anyone with domain expertise and coding ability

and churn; applications programmed to them will have to follow their changes. Because most southbound APIs represent fully standardized networking protocols that will rarely change, these will on average be more stable.

Southbound APIs. OpenFlow became prominent when startups such as Nicira (and then Big Switch Networks) first commercialized SDN products that utilized the protocol.

During 2014, other southbound mechanisms appeared, some of which, like OpenFlow, are now full-fledged protocols for defining both transport and messaging. Other mechanisms are still nascent.

Many southbound mechanisms have at least some market acceptance. The Network Configuration Protocol (Netconf) and Simple Network Management Protocol (SNMP) are perhaps the most widely accepted. OpenFlow

is next in line, followed by the Open vSwitch Database Management Protocol (OVSDb). Subsequent to these are the Extensible Messaging and Presence Protocol (XMPP), JavaScript Object Notation (JSON) over HTTPS, and other RESTful APIs, which are more transport envelopes than fully-defined protocols. Relatively new to the scene is policy-centric, declarative OpFlex, a favorite of Cisco proponents (<http://tools.ietf.org/html/draft-smith-opflex-00>).

OpenFlow's lower position in the market acceptance spectrum is not surprising, given its relatively short lifetime relative to other southbound mechanisms. Beyond version 1.0, uptake has been limited, in part because of the challenge to map OpenFlow 1.3 (beyond the minimal mandatory features) to existing ASIC-based pipelines. Another contributing factor is the lack of a standard protocol library that would suit both open source and vendor development communities—much as OpenSSL served both its communities and thus contributed to the production use of SSL protocols.

Recognizing the need for a standard and readily available library that abstracts OpenFlow intricacies, the Open Networking Foundation (ONF), which spearheads OpenFlow and SDN standardization, conducted a competition to create an open source OpenFlow driver that would make it easier to develop OpenFlow switch agents and controllers. In March 2014, ONF selected libfluid (<http://opennetworkingfoundation.github.io/libfluid>) as the winner.

Northbound APIs. At present, northbound APIs are mostly proprietary or non-standardized, depending on the SDN controller and the controller's target use cases. In the past, every open source controller defined its own basic northbound interfaces, most of which were simple, appearing as a thin layer over OpenFlow, and occasionally supporting some topology query elements.

As of September 2014, about 20 commercial SDN controllers exist, each with its own set of northbound APIs. The Plexxi controller, for example, features a workload affinity API, for communication between external systems and the controller, and a network orchestration API, which aids in translating workload requirements into network instructions.

In general, most commercial controllers provide only rudimentary northbound APIs, primarily for device-level configuration and flow programming, as opposed to offering APIs on a networkwide service level. Because network application requirements differ considerably, blends of northbound APIs should not be surprising.

In parallel to the standardization work in ONF's Northbound Interface Working Group, code from projects like ODP and OpenStack are appearing as candidate de facto northbound API standards. This trend is consistent albeit with a slight twist: the Internet Engineering Task Force's motto "running code and rough consensus"

implies that significant code has been implemented as part of a reference implementation before standardization. Moreover, some in the industry have argued that the optimal northbound APIs should at least integrate into the orchestration stack, which will foster integration into existing OSS (operations systems support) and management systems.

API pairing. With the significant differences in north- and southbound approaches and their abstraction layers, a standard layered SDN architecture is doubtful. Nevertheless, efforts like the ODP are attempting to create a unified software architecture that accommodates the different approaches.

The ODP's unification strategy centers on a model-driven service abstraction layer (MD-SAL) architecture that supports multiple abstraction models without mandating fixed abstraction layers. MD-SAL enables different northbound APIs through REST configuration (RESTConf) interfaces that use the Yang data modeling language to configure the model and manipulate state data. The resulting north- and southbound pairing will create end-to-end siloed stacks, which will persist until users see commonalities and motivate northbound API coalescence.

ROUTEFLOW: SOFTWARE-DEFINED IP ROUTING

RouteFlow is an ongoing project to glue together open source IP routing stacks and OpenFlow networks. With RouteFlow, designers use virtual network routers to create a virtualized network control plane and install the resulting packet-handling rules into the datapaths' flow tables. These rules fully represent each router's forwarding tables. As in a legacy network, the virtual routers exchange traditional network protocol messages, such as open shortest path first (OSPF), using protocols like the Border Gateway Protocol (BGP) to interconnect with external physical and legacy routers. RouteFlow is also suitable for hybrids—networks formed by SDN and legacy components.

Because IP routing is a core networking application, RouteFlow's progress in the SDN community is an excellent window on SDN advances.

Open source architectural components

As Figure 2 shows, RouteFlow leverages several open source solutions and introduces three components: RF-Client, RF-Server, and RF-Proxy. With these components and the resulting SDN stack, network architects can run Linux-based routing stacks and arbitrarily install the generated forwarding information base (FIB) into OpenFlow devices.

RF-Client is a user-space daemon that collects routing and forwarding information, such as the IP routing table, from any Linux-based routing engine. It typically runs in a

virtualized environment, such as a Linux container (LXC) or kernel virtual machine (KVM).

RF-Server is a stand-alone application that handles the system's core logic, such as event processing and component mapping. Routing-specific services are implemented as operator-tailored modules, which can use network-wide information to deliver arbitrary, high-level routing logic. Such information might include load balancing and preferred exit points, for example.

RF-Proxy is a shim proxy application atop an OpenFlow controller that handles switch interactions and uses topology discovery and monitoring to collect network state.

In a straightforward operational mode, a single data plane switch naturally associates with a single virtual router. Different modes are also possible, including

- a multiplexed mode, in which multiple virtual routers slice and control datapaths, and
- an aggregate mode,⁵ in which a single virtual router represents a datapath group, enabling a single router abstraction to use eBGP to configure domain-wide routing control platforms.

To act as a router, RouteFlow creates OpenFlow rules that match destination IP and MAC addresses, rewrite MAC addresses according to next-hop information, or employ multiple flow tables and group tables to take specific actions on incoming traffic patterns. It then installs the resulting flow-based FIB entries in OpenFlow devices. Besides serving as the data plane API, the OpenFlow protocol is the vehicle for delivering control plane messages from and to the virtualized OS's interfaces.

In line with the best cloud application design practices, RouteFlow relies on scalable, fault-tolerant data storage that centralizes the core state; the logical, physical, and protocol-specific network views; and any information base used to develop route-control applications, such as traffic forecasts, flow monitoring, and administrative policies. Either ZeroMQ or a distributed NoSQL database, such as MongoDB, can serve as the pubsub-like message-queuing interprocess communication (IPC) mechanism. The only requirement is that the IPC mechanism adhere to an extensible JSON-based RouteFlow protocol implementation to loosely couple the modules.

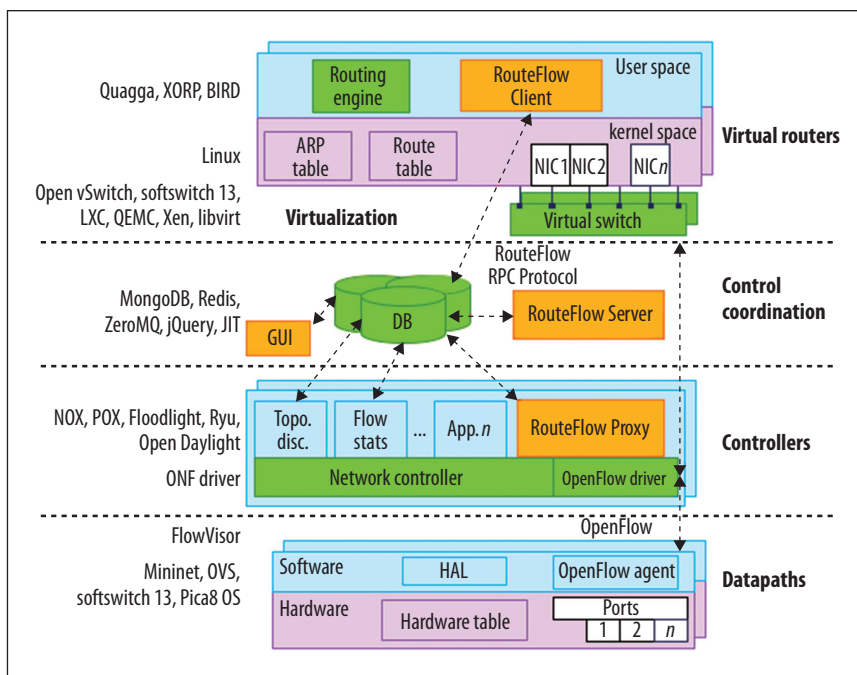


Figure 2. RouteFlow architecture. RouteFlow introduces three software components, RF-Proxy, RF-Server, and RF-Client, into a modular design. The components combine with a routing-oriented SDN stack, which is based on many widely used open source solutions (left). RPC: remote procedure call; ARP: Address Resolution Protocol; NIC: network interface card.

In essence, RouteFlow tries to follow principles that allow architectural evolution: layers of indirection, system modularity, and interface extensibility. Network architects can distribute the number of RF-Proxy and RF-Client instances and hypervisor switches arbitrarily, and RF-Protocol northbound APIs abstract the routing engine and SDN API specifics away from the core logic.

Controller independence

Any abstraction must overcome the heterogeneity of southbound OpenFlow APIs and the lack of standardized northbound APIs. The latter obstacle motivated our work on route-centric simplified OpenFlow messaging, a unifying abstraction that makes it easier to develop a shim RF-Proxy application for multiple controllers while keeping the remaining modules intact and independent of the southbound protocol version.⁸

RF-Protocol acts as an IPC that glues different modules together with a simple command/response JSON syntax northbound to RF-Clients and a subset of OpenFlow messages southbound to RF-Proxy.

The result is a hierarchical design that is independent of a specific controller and agnostic to the OpenFlow version that the forwarding devices support. Additional work will aim to enable interaction with heterogeneous OpenFlow devices, which is critical since these devices' capabilities and optional feature sets can differ significantly—an obstacle

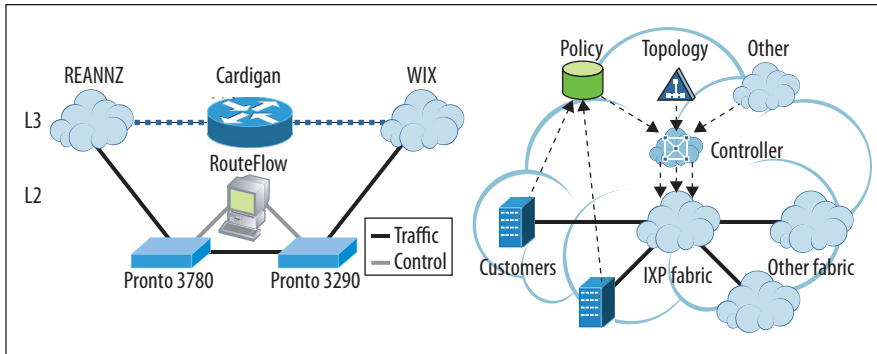


Figure 3. How Cardigan works. Running a tailored RouteFlow version, the Cardigan controller implements the logical router interconnection between the Research and Education Advanced Network New Zealand (REANNZ) and the Wellington Internet Exchange (WIX), which controls two OpenFlow switches (Pronto 3780 and Pronto 3290) that are physically interconnected through dark fiber. IXP: Internet exchange point.

already receiving attention from the ONF's Forwarding Abstractions Working Group. We also expect potentially useful IP routing-oriented APIs from related work on IETF I2RS, despite its focus on in-box routing stacks and distributed control planes.

CARDIGAN: ROUTEFLOW LIVE

Cardigan, in operation since 2014, is the code name for efforts that leverage RouteFlow to realize SDN production goals, including

- seamless replacement of deployed technology,
- zero performance penalties, and
- game-changing innovation relative to traditional operations.

Cardigan deployment⁶ began as a simple network: two OpenFlow switches in different buildings controlled by a single controller, peering the Wellington Internet Exchange (WIX) on one side and the Research and Education Advanced Network New Zealand (REANNZ) office network on the other. However, the two OpenFlow switches appear as a single router when commanded through the single, logically centralized abstraction provided by the RouteFlow control platform.

As Figure 3 shows, the system connects directly to the Internet through the WIX, and its configuration management is from a single, logically centralized point, via which a network operator can add, translate, and distribute routes to the switches. More importantly, this same paradigm extends even as additional switches are added to the network.

This network provides a platform for practical SDN research in a realistic hybrid and live deployment, peering with non-SDN devices and directly connected to the Internet. Approximately 90 organizations peer through WIX; most are IPv6 ready, handling up to 500 IPv6 and 1,000 IPv4

network announcements (<http://wix.nzix.net/peers.html>).

Although Cardigan's initial deployment size was modest, we believe that practical production deployment is the best way to boost SDN production projects. Even basic systems need to get in production as early as possible to perform actual work outside the lab and feed experiential results into the development-deployment cycle.

Cardigan uses Vandervecken (<http://github.com/routeflow/RouteFlow/tree/vandervecken>), a RouteFlow fork aimed at continual consolidation of pro-

duction-based research. Flow programming is proactive and is based on the RouteFlow IP route and Address Resolution Protocol (ARP) table computations; all other forwarding is denied by default, which limits the controller's exposure to denial-of-service attacks.

As of September 2014, each switch has 1,138 flows: eight flows tunnel control plane traffic to the controller; 98 flows describe directly connected hosts at WIX and REANNZ; and 1,032 flows represent layer 3 routes. The aggregate traffic fluctuates around hundreds of megabits per second. Changes at the Internet exchange point (IXP) provide three to four updates every 10 seconds to account for ARP timeouts and link changes. Given the deployment's live nature, the Cardigan project team has not yet conducted a deeper performance analysis.

Cardigan has recently added OpenFlow 1.3 support as a critical step to work with multiprotocol label switching (MPLS) and IPv6 and is using group tables to efficiently implement IP next-hop forwarding. Beyond L2/L3 interworking, Cardigan has achieved basic label switch router (LSR) interoperability. The Cardigan network now spans the US (ESnet), Australia (CSIRO), and New Zealand (REANNZ and VUW). Interoperability with Brazil (RNP) is planned.

NEXT STEPS

Obviously, increased SDN deployment faces technical challenges, such as scalability, performance, dependability, and product maturity,⁹ which both the academic research community and ONF industrial partners are actively addressing.

For RouteFlow, we will be adding full support of OpenFlow 1.3.4, IPv6, and RSVP-TE to the SDN routing stack. Planned experiments with various OpenFlow data plane implementations (including ASIC, NPU, SW, and hybrids) will investigate scalability of flow number, state-change rate, device number, and distance between distributed devices.

At the controller level, we are increasing RouteFlow's availability, including OpenFlow master/slave/equal role support and establishing interoperability with several additional controllers, among them On.Lab's ONOS and Open Daylight. We also recognize that RouteFlow must withstand denial-of-service attacks; it must be easier to manage than a network of separate routers, but it must also be more reliable and easier to fix. In addition to exploiting common software engineering practices, we will be leveraging SDN research on formal methods to build provably correct networks and systematically troubleshooting them.

Finally, we are researching practical questions that arise in operating multiple SDNs, including suitable SDN-to-SDN protocols. For example, how should SDNs in different administrative domains communicate beyond BGP?

We are devoting these efforts to realizing the vision of software-defined IXPs,¹⁰ in which participants can peer on the basis of topology, metrics, and policies that are far more expressive than BGP, as illustrated in Figure 3.

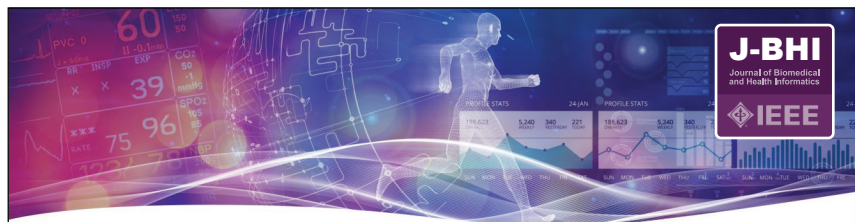
We expect the SDN horizon to radically change current network design, building, and operation, and invigorate innovation through open, vendor-agnostic architectures that provide clear decoupling of data and control planes.

Open source and SDN are critical partners in moving the networking industry forward in this change. Significant progress is already evident in the number and diversity of open source SDN projects, including practical deployments like the RouteFlow architecture and operational experiences like Cardigan. Even so, the SDN community must solve foundational standardization and API maturity issues

before widely deployable open source SDN stacks will appear in production. **□**

Acknowledgments

We thank David Meyer and Inder Monga for insightful comments and the anonymous reviewers of earlier versions of this article. We also thank Diego Kreutz for his contribution to Figure 1. This work was partially supported by the Innovation Center, Ericsson Telecomunicações S.A., Brazil.



IEEE JOURNAL OF BIOMEDICAL AND HEALTH INFORMATICS

<http://jbhi.embs.org>

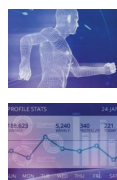
J-BHI is one of the leading journals in computer science and information systems with a strong interdisciplinary focus and biomedical and health application emphasis. J-BHI publishes papers on the following topics:

- **Bioinformatics**
- **Imaging Informatics**
- **Sensor Informatics**
- **Medical Informatics**
- **Public Health Informatics**

These are complemented by managed special issues/sections covering topics that are of strategic importance to the journal, coordinated by guest editors who are leading experts in these fields. Recent special issues include **Big Data for Health, Bioinformatics in Clinical Environments and Service Science for e-Health**.

The Journal was retitled from the IEEE Transactions on Information Technology in Biomedicine (T-ITB) in 2013. We particularly encourage large cohort studies with clearly demonstrated clinical translational values supplemented by online data sets or algorithms that can be shared by the research community.

Upcoming Special Issues:



Sensor Informatics and Quantified Self
(deadline 18 December 2014)

Biomedical and Health Informatics for Diabetes
(deadline 28 February 2015)

Visit <http://jbhi.embs.org> to subscribe to our mailing list, receive monthly J-BHI highlights, and read about our latest featured articles.

Editor-in-Chief: Guang-Zhong Yang, PhD, FREng, FIEEE
Email: jbhi-eic@embs.org
Director, The Hamlyn Centre, Imperial College London,
London SW7 2AZ, United Kingdom



References

1. D. Meyer, "The Software-Defined-Networking Research Group," *IEEE Internet Computing*, vol. 17, no. 6, 2013, pp. 84–87.
2. N. McKeown et al. "OpenFlow: Enabling Innovation in Campus Networks," *SIGCOMM Computing Comm. Rev.* vol. 38, no. 2, 2008, pp. 69–74.
3. T. Nadeau and K. Gray, *SDN: Software Defined Networks*, O'Reilly Media, 2013.
4. D. Kreutz et al., "Software-Defined Networking: A Comprehensive Survey," to be published in *Proc. IEEE*, 2015; <http://arxiv.org/abs/1406.0440>.
5. C. Rothenberg et al., "Revisiting Routing Control Platforms with the Eyes and Muscles of Software-Defined Networking," *Proc. ACM Hot Topics in Software-Defined Networking (HotSDN 12)*, 2012, pp. 13–18.
6. J. Stringer et al., "Cardigan: Deploying a Distributed Routing Fabric," *Proc. ACM Hot Topics in Software-Defined Networking (HotSDN 13)*, 2013, pp. 169–170.
7. J. Lindman et al., "Matching Open Source Software Licenses with Corresponding Business Models," *IEEE Software*, vol. 28, no. 4, 2011, pp. 31–35.
8. J. Stringer and C. Owen, "RouteMod: A Flexible Approach to Route Propagation," Victoria Univ. of Wellington, 2013; <http://wand.net.nz/~jps22/routemod.pdf>.
9. S. Yeganeh et al., "On Scalability of Software-Defined Networking," *IEEE Comm.*, vol. 51, no. 2, 2013, pp. 136–141.
10. A. Gupta et al., "SDX: A Software Defined Internet Exchange," *Proc. 2014 Ann. Conf. ACM Special Interest Group Data Comm. (SIGCOMM 14)*, 2014, pp. 551–562.

Christian Esteve Rothenberg is an assistant professor at the University of Campinas, Brazil. His research interests include IP systems, networking, and open source SDN projects, such as RouteFlow, softswitch 13, and libfluid. Rothenberg received a PhD in electrical and computer engineering from the University of Campinas. He has two international patents. Rothenberg is a member of IEEE, ACM, and the Brazilian Computing Society, and an ONF research associate. Contact him at chesteve@dca.fee.unicamp.br.

Roy Chua is cofounder and a partner of SDNCentral.com and a partner in Wiretap Ventures. His research interests include SDN, network virtualization, network security and cloud platforms. Chua received an MBA from MIT's Sloan School of Management and an MS in electrical engineering and computer science from the University of California at Berkeley. Contact him at rchua@sdncentral.com.

Josh Bailey is a software engineer at Google in Wellington, New Zealand. His research interests include large-scale,

high-performance networks and computers. Contact him at joshb@google.com.

Martin Winter is a technical lead and cofounder of the Network Device Education Foundation. His research interests include routing platforms, networked systems, and software engineering as well as OpenSourceRouting, a non-profit project to improve Quagga. Winter received a BS in computer science from Brugg-Windisch HTL. He is chair of the Open Source Working Group in RIPE (European Internet Provider Forum). Contact him at mwinter@netdef.org.

Carlos N.A. Corrêa is a PhD candidate in computer science at the Federal Fluminense University and an IT operations consultant at Unimed. Corrêa received an MS in information systems from the Federal University of Rio de Janeiro. Contact him at carlos.nilton@gmail.com.

Sidney C. de Lucena is an assistant professor of information systems at the Federal University of the State of Rio de Janeiro. His research interests include network management and SDN. De Lucena received a DSc in computer and systems engineering from the Federal University of Rio de Janeiro. He is a member of the Brazilian Computing Society. Contact him at sidney@uniriotec.br.

Marcos Rogério Salvador is a research collaborator at the University of Campinas and senior manager of innovation at the Lenovo Innovation Center, Brazil. His research interests include network virtualization, software-defined infrastructures, and cloud datacenter architectures. Salvador received a PhD in computer science from the University of Twente, the Netherlands. He is an ONF research associate. Contact him at msalvador@lenovo.com.

Thomas D. Nadeau is a Distinguished Engineer at Brocade and Chief Architect of Open Source in the company's Software Business Unit. His research interests include building commercial open source–based products, SDN, NFV and open-source projects such as Open Daylight and OpenStack. Nadeau received an MSc from the University of Massachusetts in Lowell, where he is an adjunct professor of computer science. He is cochair of the IETF NETMOD Working Group and coauthor of *SDN: Software Defined Networks, An Authoritative Review of Network Programmability Technologies* (O'Reilly, 2014). Contact him at tnadeau@brocade.com. 



Selected CS articles and columns are available for free at <http://ComputingNow.computer.org>.